CSA0506 - DATABASE MANAGEMENT SYSTEMS

CO4 | ASSESSMENT TOOL 2 - Comparative Analysis

Name:	Nokeshwaran	**Reg No:**	192511007	**Branch:**	B.E CSE

Q1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Aspect	Sequential File	Heap File
Record order	Records stored in sorted order by key	Records stored in insertion order (unordered)
Insertion	Slow record must be placed in correct sorted position ($O(n)$ shift)	Fast record appended at end ($O(1)$)
Deletion	Requires shifting records to close the gap, or tombstoning	Simple mark as deleted or remove, no reordering needed
Retrieval (search)	Fast binary search possible, $O(\log n)$	Slow linear scan required, $O(n)$
Range queries	Efficient, since records are already ordered	Inefficient — must scan and sort

Summary: Sequential file organization trades off insertion/deletion speed for fast, ordered retrieval, making it ideal for read-heavy applications. Heap files trade off retrieval speed for very fast insertion, suiting write-heavy applications where ordering isn't required (e.g., log files, staging tables).

Q2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Aspect	Clustered Index	Non-Clustered Index
Data arrangement	Table rows physically sorted/stored according to index key	Separate structure with pointers to actual row locations
Number per table	Only one per table	Multiple allowed per table
Retrieval speed	Very fast for range queries	Slower needs extra pointer lookup
Storage overhead	Low	Higher (separate structure stored)

Example: An *Employees* table clustered on `employee_id` physically stores rows in ID order — ideal for `WHERE employee_id BETWEEN 100 AND 200`. A non-clustered index on `employee_name` in the same table allows fast name lookups without altering physical row order, but each lookup needs an extra step to fetch the actual row.

Q3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

Aspect	Primary Index	Secondary Index
Based on	Primary key of a sorted data file	Any non-primary attribute
Data file order	Must be physically sorted on indexed field	Independent of index
Density	Usually sparse	Usually dense
Storage requirement	Lower (fewer entries)	Higher (entry per record)
Search performance	Very fast sparse index + sequential order	Slightly slower dense index, no physical correlation

Summary: Primary indexing is storage-efficient because it leverages physical file ordering. Secondary indexing needs more storage since it must index every record individually, but it enables fast lookups on fields the file isn't physically sorted by.

Q4. Analyze the differences between static hashing and dynamic hashing in database systems.

Aspect	Static Hashing	Dynamic Hashing
Number of buckets	Fixed at creation	Grows/shrinks with data volume
Handling of growth	Poor — overflow chains or full rehash needed	Handles growth by splitting only affected buckets
Performance over time	Degrades as collisions increase	Stays consistently efficient ($O(1)$ average)
Complexity	Simple	More complex (directory management)

Summary: Static hashing suits databases with a known, stable size. Dynamic hashing (extendible/linear) adapts bucket count on demand, avoiding performance degradation as data grows, at the cost of extra directory-management overhead.

Q5 (Bloom's Level 4 - Analyze): Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Aspect	B-Tree	B+ Tree
Data storage	Data stored in both internal and leaf nodes	Data stored only in leaf nodes
Leaf linking	Not linked	Leaves linked sequentially
Range queries	Slower — repeated traversal needed	Fast — sequential scan across linked leaves
Search consistency	Varies by node depth	Uniform, since all data is at leaf level

Summary: For large-scale databases, B+ Trees are preferred: leaf-level linking enables efficient range queries and sequential scans, and search time is predictable since every record sits at the leaf level.

Q6 (Bloom's Level 4 - Analyze): Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Aspect	Linear Hashing	Extendible Hashing
Directory structure	No directory — buckets split in predetermined order	Uses a directory of pointers to buckets
Bucket splitting	Round-robin, not necessarily the overflowing bucket	Splits directly at the overflowing bucket
Growth mechanism	Gradual, controlled by a split pointer	Directory doubles when local depth equals global depth
Overhead	Lower — no directory	Higher — directory maintenance

Summary: Linear hashing avoids directory overhead and grows predictably. Extendible hashing resolves overflow immediately at the source bucket but risks sudden memory jumps from directory doubling — a scalability vs. simplicity trade-off.

Q7 (Bloom's Level 4 - Analyze): Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Aspect	Dense Index	Sparse Index
Index entries	One per record	One per block
Memory usage	High — grows with number of records	Low — grows with number of blocks
Access speed	Faster — direct pointer to exact record	Slightly slower — needs in-block scan after locating block
File requirement	Works on sorted or unsorted files	Requires sorted data file

Summary: Dense indexing gives faster direct access at high memory cost. Sparse indexing sharply reduces memory usage in exchange for a small extra in-block search step, making it preferable for very large sorted files.

Q8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Aspect	Indexed Sequential	Direct (Hashed)
Access method	Index + sequential storage	Direct computation via hash function
Retrieval speed	Fast for both sequential and random access	Very fast ($O(1)$ avg) for exact-match only
Range queries	Well supported (sorted data)	Poorly supported (no ordering)
Insertion overhead	Higher — sorted order and index maintained	Lower — direct placement unless collisions occur

Summary: Advantage of indexed sequential: supports both range and sequential access efficiently. Limitation: costly insert/delete maintenance and possible overflow areas. Advantage of direct: near-instant exact-match lookup. Limitation: no ordering, so range queries and sorted reports are very inefficient.

Q9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Aspect	Single-Level Index	Multi-Level Index
Structure	One flat index over the data file	Hierarchy of indexes (index over an index)
Search complexity	$O(\log n)$, but index can grow very large	$O(\log_b n)$ with fan-out b — far fewer comparisons
Memory / I/O	Entire index may not fit in memory for large files	Top levels stay small/in-memory, reducing disk I/O
Scalability	Degrades for very large files	Scales well — basis of B-Tree/B+ Tree design

Summary: Single-level indexing is adequate for moderate file sizes, but as data grows the flat index itself becomes too large to search efficiently. Multi-level indexing solves this by layering indexes hierarchically, reducing disk I/O and making it the standard for large-scale retrieval.

Q10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

For exact-match queries:

- Hashing computes the bucket location directly, giving $O(1)$ average-case lookup regardless of data size.
- Indexing (e.g., B+ Tree) needs $O(\log n)$ comparisons to traverse the tree — fast, but inherently slower than direct hash computation.
- **Verdict:** Hashing performs better for exact-match/point queries.

For range queries:

- Hashing performs poorly, since hash functions scatter keys pseudo-randomly across buckets, giving no way to retrieve a contiguous range without scanning everything.
- Indexing (especially B+ Trees with linked leaves) excels — once the start key is located, sequential traversal of linked leaves retrieves the entire range efficiently.
- **Verdict:** Indexing performs better for range queries.

Overall Judgment: Neither technique is universally superior; the right choice depends on query workload. Hashing (static, dynamic, or extendible) should be chosen when the workload is dominated by exact-match lookups (e.g., authentication, key-value access). Tree-based indexing (B-Tree/B+ Tree) should be chosen when range queries or sorted retrieval matter (e.g., financial reports, date-range searches). In practice, production databases often combine both — hashing for primary-key lookups and B+ Tree indexes for range and multi-attribute queries — because this reflects the complementary strengths of each: hashing optimizes for speed of exact match, indexing optimizes for flexibility and ordering.